

# Recurrent Graph Transformers for the Maximum Independent Set Problem

## 1 Approach

**Problem.** Given an undirected graph  $G = (V, E)$ , the *Maximum Independent Set* (MIS) problem asks for the largest  $S \subseteq V$  such that no two nodes in  $S$  are adjacent. MIS is NP-hard. We treat it as a per-node binary classification task: the model predicts  $p_v \in [0, 1]$ , the probability that node  $v$  belongs to an optimal independent set, and a deterministic decoder turns these scores into a feasible set.

**Architecture.** The model is a *recurrent graph transformer* (**GraphTransformerTRM**,  $\approx 1.55$  M parameters) that couples a GPS message-passing/attention backbone with a TRM-style (Tiny Recursive Model) reasoning loop:

- **Input features.** Node features  $x_v = [1, \deg_v / (n-1)]$  augmented with a 16-dimensional positional encoding (random-walk / Laplacian PE), giving a 12-dim embedding after a linear+LayerNorm stem.
- **GPS layer** (hidden = 256, 2 layers, 2 heads): each layer combines a *local* GIN message-passing branch with a *global* multi-head self-attention branch, so a node aggregates both neighbourhood and long-range information.
- **Recurrent reasoning.** A latent state  $z$  and an output state  $y$  are refined by  $H=2$  outer cycles of  $L=6$  latent updates ( $z \leftarrow \text{GPS}(x, y, z)$ , then  $y \leftarrow \text{head}(y, z)$ ). The first  $H-1$  cycles run under **no\_grad** and only the final cycle backpropagates, i.e. truncated back-propagation through a 12-step reasoning trajectory. This lets a small network “think” iteratively rather than scale parameters.

**Training.** Fully supervised. The loss combines a class-weighted binary cross-entropy with a soft feasibility penalty,

$$\mathcal{L} = \underbrace{\text{BCE}_w(\sigma(z), y)}_{\text{node label}} + \lambda \underbrace{\frac{1}{2|E|} \sum_{(u,v) \in E} p_u p_v}_{\text{edge violations}}, \quad \lambda = 2,$$

where the feasibility term directly punishes selecting both endpoints of an edge. For the small graphs we use *multi-MIS soft labels*: Gurobi enumerates up to 16 strictly-optimal MISs (`PoolSearchMode=2`, `PoolGap=0`) and the target  $y_v$  is the *fraction* of optimal solutions that contain  $v$  — a calibrated target whose Bayes-optimal BCE is  $> 0$  (e.g. 0.19 at  $n=50$ ). Optimisation: AdamW (wd = 0.05), OneCycleLR (`max_lr` =  $3 \times 10^{-4}$ , 10% warm-up, cosine anneal), `bf16` autocast, dropout 0.15, with early stopping on the held-out eval BCE.

**Evaluation metric.** At eval time the soft scores are decoded by a *greedy* rule (sort by  $p_v$ , add a node if no selected neighbour blocks it), which is *always feasible*. The headline number is the post-processed approximation ratio

$$\text{pp-AR} = \frac{|S_{\text{greedy}}|}{|S^*|} \in [0, 1], \quad \text{pp-AR} = 1 \Leftrightarrow \text{optimal},$$

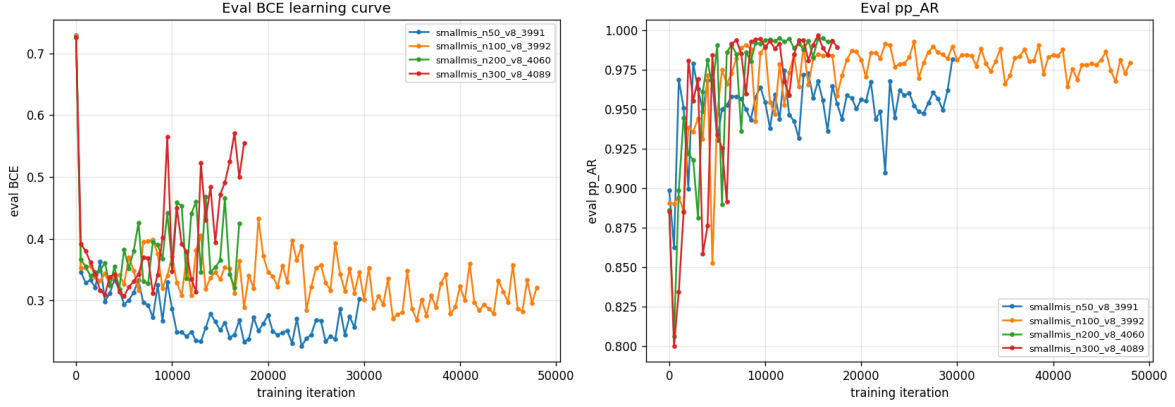
reported as a per-graph mean. Because greedy decoding guarantees feasibility, post-processed feasibility (pp-feas) is 1.0 in every run.

## 2 Experiments

We study two regimes: (i) a controlled *size-scaling* study on Erdős–Rényi (ER) graphs of fixed average degree  $d = 7.35$  for  $n \in \{50, 100, 200, 300\}$  (each with a held-out test split, Gurobi-exact labels), and (ii) two large *benchmark* datasets from the DIFUSCO suite: **SATLIB** (SAT→MIS conflict graphs,  $\sim 1400$  nodes) and **ER-700-800** (dense ER graphs,  $n \in [700, 800]$ ,  $\bar{d} \approx 112$ ), labelled by KaMIS.

### 2.1 Learning curves and scaling

Across all four small sizes the model exhibits a textbook descent: eval BCE falls from the random-init value  $\approx 0.73$  ( $\approx \ln 2$ ) to 0.23–0.32, while pp-AR rises from  $\approx 0.886$  at initialisation to  $\geq 0.98$  (Fig. 1). Larger graphs converge *faster* in iterations ( $n=200/300$  peak around iter 11k–15k) but also overfit sooner because their training pools are smaller (12k vs. 50k graphs); early stopping on eval BCE selects the best checkpoint in each case.



**Figure 1:** Held-out learning curves for  $n \in \{50, 100, 200, 300\}$  ER graphs. Left: eval BCE (lower is better). Right: post-processed approximation ratio pp-AR (higher is better). Uniform eval cadence every 500 iterations, plus an iteration-0 random-init baseline.

**Table 1:** Main results. Small ER rows are size-scaling runs (best checkpoint by pp-AR); the bottom block are large-scale benchmarks.  $q$  is the mean MIS fraction  $|S^*|/n$ . Node-level F1/prec/rec are at the same checkpoint.

| Dataset      | $n$         | $q$  | init BCE | best BCE     | pp-AR        | F1    | prec  | rec   | stop iter |
|--------------|-------------|------|----------|--------------|--------------|-------|-------|-------|-----------|
| ER $d=7.35$  | 50          | 0.32 | 0.729    | <b>0.226</b> | <b>0.982</b> | 0.847 | 0.848 | 0.847 | 29.5k     |
| ER $d=7.35$  | 100         | 0.34 | 0.728    | <b>0.268</b> | <b>0.993</b> | 0.831 | 0.825 | 0.837 | 48.0k     |
| ER $d=7.35$  | 200         | 0.36 | 0.726    | <b>0.319</b> | <b>0.995</b> | 0.820 | 0.806 | 0.835 | 17.0k     |
| ER $d=7.35$  | 300         | 0.36 | 0.725    | <b>0.307</b> | <b>0.997</b> | 0.830 | 0.819 | 0.840 | 17.5k     |
| SATLIB (CBS) | $\sim 1400$ | 0.33 | —        | 0.82         | <b>0.998</b> | 0.644 | 0.555 | 0.769 | 400k      |
| ER-700-800   | 700–800     | 0.06 | —        | —            | <b>0.903</b> | 0.201 | 0.122 | 0.572 | 325k      |

## 2.2 Confusion matrices (node level)

Table 2 reports the node-classification confusion matrix at each best checkpoint, derived exactly from the logged precision/recall and the measured MIS prevalence  $q$  (entries are % of all nodes;  $\text{Acc} = \text{TP} + \text{TN}$ ). The small-ER models are well-balanced (false positives  $\approx$  false negatives,  $\approx 90\%$  node accuracy). The benchmarks tell a different story: both *over-predict* membership (large FP), yet the greedy decoder still recovers near-optimal *sets* — the node-level metric and the MIS objective decouple (see Insights).

**Table 2:** Node-level confusion matrices (% of all nodes) at the best checkpoint.

| Dataset ( $n$ ) | TP   | FP   | FN  | TN   | Acc         |
|-----------------|------|------|-----|------|-------------|
| ER $n=50$       | 27.4 | 4.9  | 5.0 | 62.7 | <b>90.1</b> |
| ER $n=100$      | 28.6 | 6.1  | 5.6 | 59.8 | <b>88.4</b> |
| ER $n=200$      | 29.9 | 7.2  | 5.9 | 57.0 | <b>86.9</b> |
| ER $n=300$      | 30.6 | 6.8  | 5.8 | 56.8 | <b>87.4</b> |
| SATLIB          | 25.6 | 20.6 | 7.7 | 46.1 | 71.7        |
| ER-700-800      | 3.4  | 24.7 | 2.6 | 69.3 | 72.7        |

## 2.3 When it stopped working

**Statistical limit — dense graphs.** On ER-700-800 (avg. degree  $\approx 112$ , MIS fraction only  $q \approx 0.06$ ) the node classifier collapses:  $\text{F1} = 0.20$ , precision = 0.12. The model essentially learns a *ranking* rather than a calibrated classifier; greedy decoding rescues it to pp-AR = 0.90 (vs. a degree-heuristic baseline of pp-AR = 0.60), but the BCE *rises* during training even as pp-AR improves — a clear sign that the per-node objective is mis-aligned with the MIS objective at high density.

**Hardware/scaling limit — ER at  $N=9000$ .** Training the dense ER-700-800 distribution with a 9000-graph in-memory pool ran out of GPU memory (CUDA OutOfMemoryError, 31.4 GB card) inside the GIN message-passing `index_select`, in two independent attempts. The same  $N=9000$  budget *succeeded* on SATLIB (sparser conflict graphs) and reached the best result of the whole study, pp-AR = 0.998. The bottleneck is therefore the *edge count* of the batched graph (dense ER has  $\sim 40\times$  more edges per node than SATLIB), not the number of graphs.

### 3 Key Insights

1. **The model genuinely learns at small/medium scale.** Every  $n \in \{50, \dots, 300\}$  run shows monotone BCE descent *and* monotone pp-AR ascent on held-out data, reaching pp-AR  $\in [0.98, 0.997]$  with  $\approx 90\%$  node accuracy — not memorisation, since train and test use disjoint seed ranges.
2. **Node-level loss  $\neq$  MIS quality.** The decisive metric is the post-processed approximation ratio, not BCE/F1. On SATLIB the model has only F1= 0.64 yet pp-AR = 0.998; on dense ER, F1= 0.20 yet pp-AR = 0.90. A weak but *correctly ordered* score, combined with a feasibility-preserving greedy decoder, is enough to obtain near-optimal sets. Optimising or early-stopping on BCE alone is therefore misleading for this task.
3. **Graph density is the real difficulty axis, not graph size.** Larger *sparse* graphs (SATLIB,  $\sim 1400$  nodes) are *easier* than smaller *dense* graphs (ER-700-800). Performance and even trainability (OOM) track edge count, because both attention cost and label sparsity ( $q$  from 0.36 down to 0.06) scale with density.
4. **Feasibility is free, optimality is the target.** The greedy decoder makes pp-feas = 1.0 everywhere, so the feasibility loss mainly shapes the ranking; all remaining effort goes into closing the optimality gap.
5. **Recurrent depth substitutes for parameters.** A single 1.55 M model, unrolled for 12 reasoning steps, suffices across two orders of magnitude in graph size — supporting the TRM premise that iterated computation can replace width/depth for combinatorial reasoning.

*Reproducibility.* Small-ER runs: SLURM jobs 3991/3992/4060/4089 (`slurm/smallmis_n{50,100,200,300}_v8.slurm`), trainer `experiments/overfit_sl_pe.py`, data `dataset/build_mis_dataset_multilabel.py`. Benchmarks: SATLIB job 3025, ER-700-800 job 2506. Confusion matrices computed from logged precision/recall and measured prevalence  $q$ ; figures from `visualizations/plot_curves.py`.